LABORATORY 07 – Basic Infrastructure and Network Layer

AUTORS

Andrés Felipe Chavarro Plazas

David Santiago Espinosa Rojas

TEACHER

Diego Alejandro Murcia Cespedes

COMPUTER NETWORKS

GROUP 01

Bogota, Colombia

April 27, 2025

OBJETIVE

Continue learning about the operation of operating systems and network services. Install network management tools and configure routers with static routing, using specialized hardware and software.

MATERIALS & TOOLS

- Laboratory computers with internet access.
- Virtualization software.
- Packet Tracer & Wireshark.
- Routers & switches.

INTRODUCTION

In today's environment, an organization's technological infrastructure heavily relies on the proper functioning of its network services and operating systems. These components enable communication between devices, access to distributed services, and efficient management of both physical and virtualized resources.

This laboratory aims to provide experience in the installation, configuration, and monitoring of network tools and devices. Students will work with different operating systems, virtualization platforms, cloud services, and physical devices such as routers and switches to simulate real world network infrastructure scenarios.

Throughout the lab, we will carry out a variety of practical activities. These include exploring and scripting with network diagnostic commands, creating shell programs for port scanning, and setting up network monitoring servers using different operating systems. In addition, they will access and configure routers through console connections, assign IP addresses, establish serial connections, and implement static routing to enable communication across multiple networks. Also, we will deploy a basic web application using Azure and monitor its performance using Application Insights. All configurations and results are going to be properly documented and presented for evaluation.

THEORETICAL FRAMEWORK

A fundamental part of the network practice involves the use of diagnostic and monitoring tools such as netstat, vnstat, route, and ethtool. These commands allow administrators to observe and interpret network traffic, interface statistics, routing tables, and device configurations. Their proper usage is essential for understanding the current state of a network and for troubleshooting connectivity issues.

Another important component is the configuration of routers, which includes assigning IP addresses, setting up interface descriptions, configuring passwords for different access levels (console, telnet, privileged mode), and synchronizing messages. This setup provides the bases for secure and organized network device administration. Additionally, understanding the router's boot process and memory types such as NVRAM, RAM, ROM, and flash memory, is necessary when performing initial setups or password recovery.

Also, routers do the implementation of static routing. This involves manually defining the paths that data packets should take to reach networks that are not directly connected. Static routes require careful planning and understanding of subnetting, IP addressing, and output interfaces. It is also essential to verify routing functionality using tools like ping and traceroute.

Network monitoring through SNMP platforms is another focus. These platforms provide realtime data about network performance, device health, and service status. By deploying monitoring solutions across different operating systems.

Finally, the lab introduces students to cloud-based network administration via Microsoft Azure. By deploying and monitoring a simple web application, students explore how cloud platforms offer integrated monitoring tools like Application Insights. This connects the concepts of infrastructure, performance metrics, and application-layer monitoring, tying back to the underlying network and transport layers responsible for data transmission and reliability.

BASE SOFTWARE INSTALLATION

In this section, we created Shell scripts to display network command outputs and check if ports are open and which services are running.

OTHER USEFUL COMMANDS

Before we begin, we need to understand what the `***netstat***`, `***vnstat***`, `***route***`, and `***ethtool***` commands are and what they are used for. These are fundamental tools for network administration and diagnostics in Unix-like operating systems and other environments. Each provides specific and valuable information about the status and behavior of the network:

- `***netstat***` allows you to view active network connections, routing tables, interface statistics, TCP/UDP connections, and listening ports. It is useful for identifying services in use, detecting suspicious connections, and verifying network performance.
- `***vnstat***` is a tool for monitoring network traffic. Unlike other commands, it relies on kernel statistics and allows you to view bandwidth consumption by day, week, or month, which is useful for historical analysis and usage control.
- `***route***` displays the system's routing table. It allows you to see how packets are routed to different networks and hosts and also allows you to manually add or remove routes. It is crucial for configuring the path that data follows within a network.
- `***ethtool*** ` allows you to view and modify parameters for Ethernet network interfaces. It provides information such as connection speed, link status, error statistics, and more. It also allows you to change settings such as auto-negotiation or duplex mode.

Now that we understand the commands used to obtain information from Unix systems, we will create a shell script that allows the user to choose between different data obtained by the commands viewed, using an interactive menu.

```bash
#!/bin/bash

menu() {
	clear
	echo "1. mostrar informacion de read (netstat)"
	echo "2. mostrar informacion del uso de red (iftop)"
	echo "3. mostrar informacion de enrutamiento (route)"
	echo "4. mostrar informacion de interfaz (ethtool)"
	echo "5. salir"
	echo "Seleccione una opcion"
	echo ""
}

option_netstat() {
	netstat -nr
	echo "Preciona claulquier tecla para continuar"
	read
}

option_iftop() {
	iftop
	echo "Preciona claulquier tecla para continuar"
	read
}
```

```bash
option_route() {
	route -n
	echo "Preciona claulquier tecla para continuar"
	read
}

option_ethtool() {
	ethtool eth0
	echo "Preciona claulquier tecla para continuar"
	read
}

while true; do
	menu
	read option
	case $option in
		1) option_netstat ;;
		2) option_iftop ;;
		3) option_route ;;
		4) option_ethtool ;;
		5) echo "Saliendo..."; break ;;
		*) echo "Seleccione otra opcion" ;;
	esac
done
```

This small program can be validated in the following way: by viewing the content of each of the option:

- Option 1:

```
1. mostrar informacion de read (netstat)
2. mostrar informacion del uso de red (iftop)
3. mostrar informacion de enrutamiento (route)
4. mostrar informacion de interfaz (ethtool)
5. salir
Seleccione una opcion

1
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
0.0.0.0         10.2.65.1       0.0.0.0         UG        0 0          0 eth0
10.2.0.0        0.0.0.0         255.255.0.0     U         0 0          0 eth0
127.0.0.0       0.0.0.0         255.0.0.0       U         0 0          0 lo
Preciona claulquier tecla para continuar
```

- Option 2:

```
 Listening on eth0 12.5Kb          25.0Kb          37.5Kb          50.0Kb      62.5Kb

TX:            cum:      0B   peak:      0b               rates:      0b      0b      0b
RX:                      0B              0b                           0b      0b      0b
TOTAL:                   0B              0b                           0b      0b      0b
```

- Option 3:

```
1. mostrar informacion de read (netstat)
2. mostrar informacion del uso de red (iftop)
3. mostrar informacion de enrutamiento (route)
4. mostrar informacion de interfaz (ethtool)
5. salir
Seleccione una opcion

3
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.2.65.1       0.0.0.0         UG    0      0        0 eth0
10.2.0.0        0.0.0.0         255.255.0.0     U     0      0        0 eth0
127.0.0.0       0.0.0.0         255.0.0.0       U     0      0        0 lo
Preciona claulquier tecla para continuar
```

- Option 4:

```
        Supported link modes:   10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Supported pause frame use: No
        Supports auto-negotiation: Yes
        Supported FEC modes: Not reported
        Advertised link modes:  10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Advertised pause frame use: No
        Advertised auto-negotiation: Yes
        Advertised FEC modes: Not reported
        Speed: 1000Mb/s
        Duplex: Full
        Auto-negotiation: on
        Port: Twisted Pair
        PHYAD: 0
        Transceiver: internal
        MDI-X: off (auto)
        Supports Wake-on: d
        Wake-on: d
        Current message level: 0x00000007 (7)
                               drv probe link
        Link detected: yes
Preciona claulquier tecla para continuar
```

On the other hand, we are going to create another shell script to validate if a port is being used by any service within our system, resulting in the following program with its respective operating validations:

```
#!/bin/bash

host="$1"
port="$2"

if nc -z "$host" "$port" 2>/dev/null; then
        echo "El puerto $port esta ABIERTO"
else
        echo "El puerto $port esta CERRADO o inaccesible"
fi
```

```
root@darkstar:~/Lab7# ./port.sh localhost 9999
El puerto 9999 esta CERRADO o inaccesible
root@darkstar:~/Lab7# ./port.sh localhost 80
El puerto 80 esta ABIERTO
root@darkstar:~/Lab7# ./port.sh localhost 22
El puerto 22 esta ABIERTO
root@darkstar:~/Lab7# ./port.sh localhost 2200
El puerto 2200 esta CERRADO o inaccesible
root@darkstar:~/Lab7#
```

NETWORK MANAGEMENT

In this section, we focus on network management and monitoring. We set up a server to monitor the status of network devices, such as CPU usage, disk space, network performance, and memory. We have a dedicated service for each system to display the monitoring of all our virtual machines and show the data in real time.

To begin, we need to install the necessary packages to run the SNMP service on Slackware and NetBSD machines. This protocol allows us to connect an external agent to monitor the device. In the case of Slackware, we have to install the necessary package with the command `***slackpkg install net-snmp***`. In our case, this package came by default when we installed the virtual machine.

```
root@darkstar:~/Lab7# snmpd -v

NET-SNMP version:  5.9.1
Web:               http://www.net-snmp.org/
Email:             net-snmp-coders@lists.sourceforge.net

root@darkstar:~/Lab7#
```

```
root@darkstar:~/Lab7# slackpkg install net-snmp

Looking for net-snmp in package list. Please wait... DONE

No packages match the pattern for install. Try:

        /usr/sbin/slackpkg reinstall|upgrade


root@darkstar:~/Lab7# clear
```

With the package ready, we need to start the service by running the control script. To do this, we try to run the command `***/etc/rc.d/rc.snmd start***`, but it tells us we don't have sufficient permissions to run the script. To fix this, we'll grant it execution permissions with the command `***chmod +x /etc/rc.d/rc.snmd***` so that when we try the first command again, it will start the target service.

```
root@darkstar:/etc/rc.d# /etc/rc.d/rc.snmpd start
bash: /etc/rc.d/rc.snmpd: Permission denied
root@darkstar:/etc/rc.d# chmod +x /etc/rc.d/rc.snmpd
root@darkstar:/etc/rc.d# /etc/rc.d/rc.snmpd start
Starting snmpd:  /usr/sbin/snmpd -A -p /var/run/snmpd -a -c /etc/snmp/snmpd.conf
root@darkstar:/etc/rc.d# 
```

Now, we'll need to add some service configurations to allow connection from other devices. We have to go to the document in `***/etc/snmp/snmp.conf***`, where we find the section with the example of the public connection map. Then, we enter the IP of the machine and the IP of the device that will have the analysis software.

```
# community or split out the write access to a different community and
# restrict it to your local network.
# Also remember to comment the syslocation and syscontact parameters later as
# otherwise they are still read only (see FAQ for net-snmp).
#

# First, map the community name "public" into a "security name"
#       sec.name        source          community
com2sec notConfigUser   default         public

rocommunity public 10.2.77.29
rocommunity public 10.2.77.27


# Second, map the security name into a group name:
#       groupName       securityModel   securityName
#group   notConfigGroup  v1              notConfigUser
#group   notConfigGroup  v2c             notConfigUser

# Third, create a view for us to let the group have rights to:
# Open up the whole tree for ro, make the RFC 1213 required ones rw.
#       name            incl/excl       subtree mask(optional)
#view    roview          included        .1
"/etc/snmp/snmpd.conf" 459L, 18640B written                        113,0-1       23%
```

At this point, we have Slackware partially configured, but now we'll configure NetBSD in a very similar way. First, we'll verify that we don't have the service, and if not, we'll download it with the command `***pkgin install net-snmp***`.

```
net# svcs -a | grep snmp
-sh: svcs: not found
net# pkg_info | grep -i snmp
net# pkgin install net-snmp
pkg_summary.gz                                    72% 4143KB 293.8KB/s   00:05 ETA
```

Now we reconfigure the service at the moment it finishes downloading and starting with the command `service smnp start`, since it is the same configuration as Slackware.

```
  GNU nano 2.9.3              /etc/net-snmp/snmpd.conf

rocommunity public 192.168.0.15
rwcommunity private
agentAddress udp:161
access  notConfigGroup ""       any     noauth  exact   systemview none
none
view    systemview      included .1     80
```

Now we can see the information of the netbsd machine from slackware, using the command `snmpwalk -v2c -c public 10.2.77.29 memory` for memory, for example, but we can controll all aspects of the sistem.

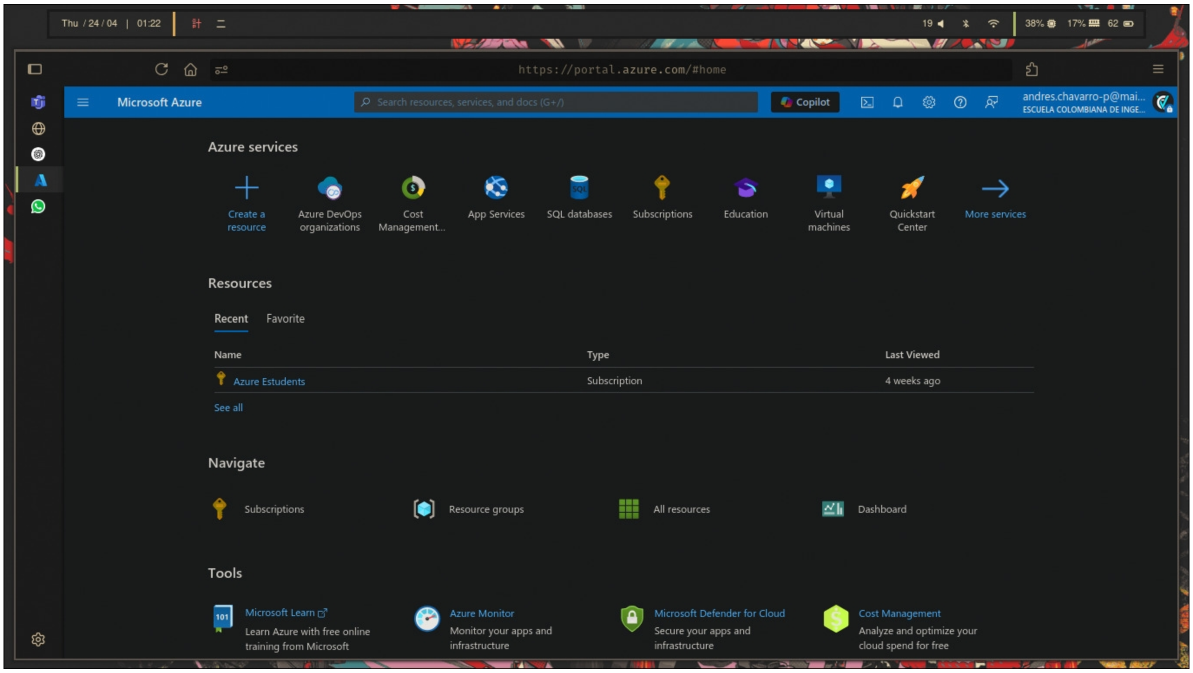
```
root@darkstar:~# snmpwalk -v2c -c public 192.168.0.21 memory
UCD-SNMP-MIB::memIndex.0 = INTEGER: 0
UCD-SNMP-MIB::memErrorName.0 = STRING: swap
UCD-SNMP-MIB::memTotalSwap.0 = INTEGER: 1048572 kB
UCD-SNMP-MIB::memAvailSwap.0 = INTEGER: 1048572 kB
UCD-SNMP-MIB::memTotalReal.0 = INTEGER: 4193852 kB
UCD-SNMP-MIB::memAvailReal.0 = INTEGER: 2555520 kB
UCD-SNMP-MIB::memTotalFree.0 = INTEGER: 3465456 kB
UCD-SNMP-MIB::memMinimumSwap.0 = INTEGER: 16000 kB
UCD-SNMP-MIB::memSwapError.0 = INTEGER: noError(0)
UCD-SNMP-MIB::memSwapErrorMsg.0 = STRING:
```

```
SNMPv2-MIB::sysORID.10 = OID: NOTIFICATION-LOG-MIB::notificationLogMIB
SNMPv2-MIB::sysORDescr.1 = STRING: The SNMP Management Architecture MIB.
SNMPv2-MIB::sysORDescr.2 = STRING: The MIB for Message Processing and Dispatchin
g.
SNMPv2-MIB::sysORDescr.3 = STRING: The management information definitions for th
e SNMP User-based Security Model.
SNMPv2-MIB::sysORDescr.4 = STRING: The MIB module for SNMPv2 entities
SNMPv2-MIB::sysORDescr.5 = STRING: View-based Access Control Model for SNMP.
SNMPv2-MIB::sysORDescr.6 = STRING: The MIB module for managing TCP implementatio
ns
SNMPv2-MIB::sysORDescr.7 = STRING: The MIB module for managing UDP implementatio
ns
SNMPv2-MIB::sysORDescr.8 = STRING: The MIB module for managing IP and ICMP imple
mentations
SNMPv2-MIB::sysORDescr.9 = STRING: The MIB modules for managing SNMP Notificatio
n, plus filtering.
SNMPv2-MIB::sysORDescr.10 = STRING: The MIB module for logging SNMP Notification
s.
SNMPv2-MIB::sysORUpTime.1 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.2 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.3 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.4 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.5 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.6 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.7 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.8 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.9 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORUpTime.10 = Timeticks: (0) 0:00:00.00
HOST-RESOURCES-MIB::hrSystemUptime.0 = Timeticks: (492607) 1:22:06.07
HOST-RESOURCES-MIB::hrSystemUptime.0 = No more variables left in this MIB View (
It is past the end of the MIB tree)
```

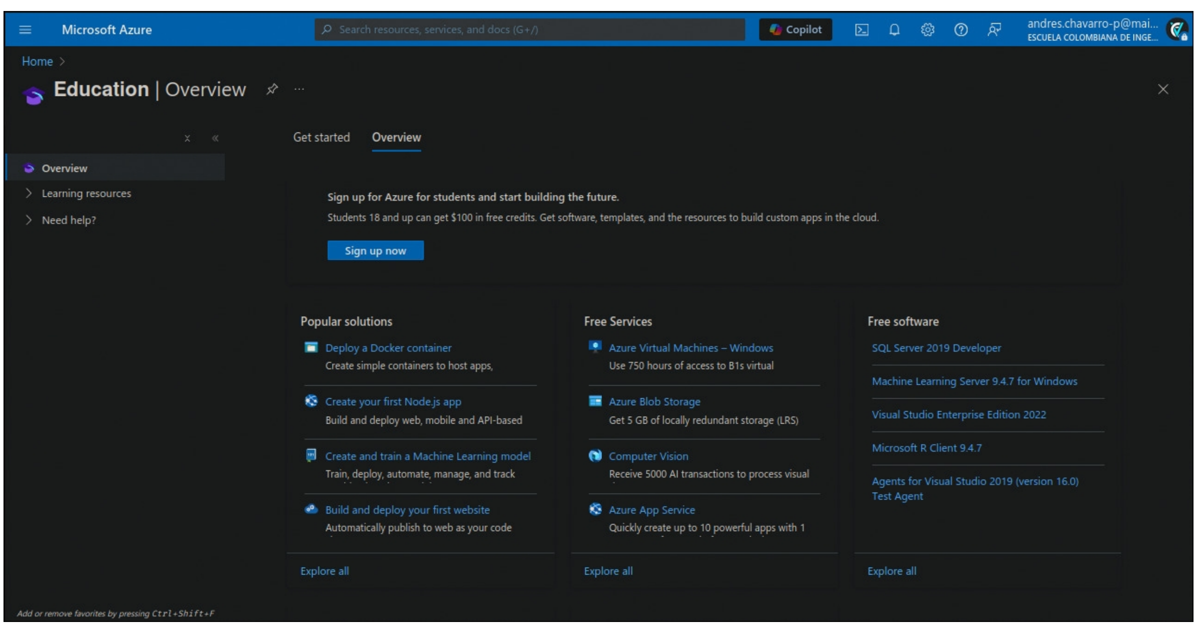
NETWORK ADMINISTRATION – AZURE

For this part, we'll use Azure to set up a Web Application and utilize the monitoring tools provided by this cloud service. To do so, follow these steps:

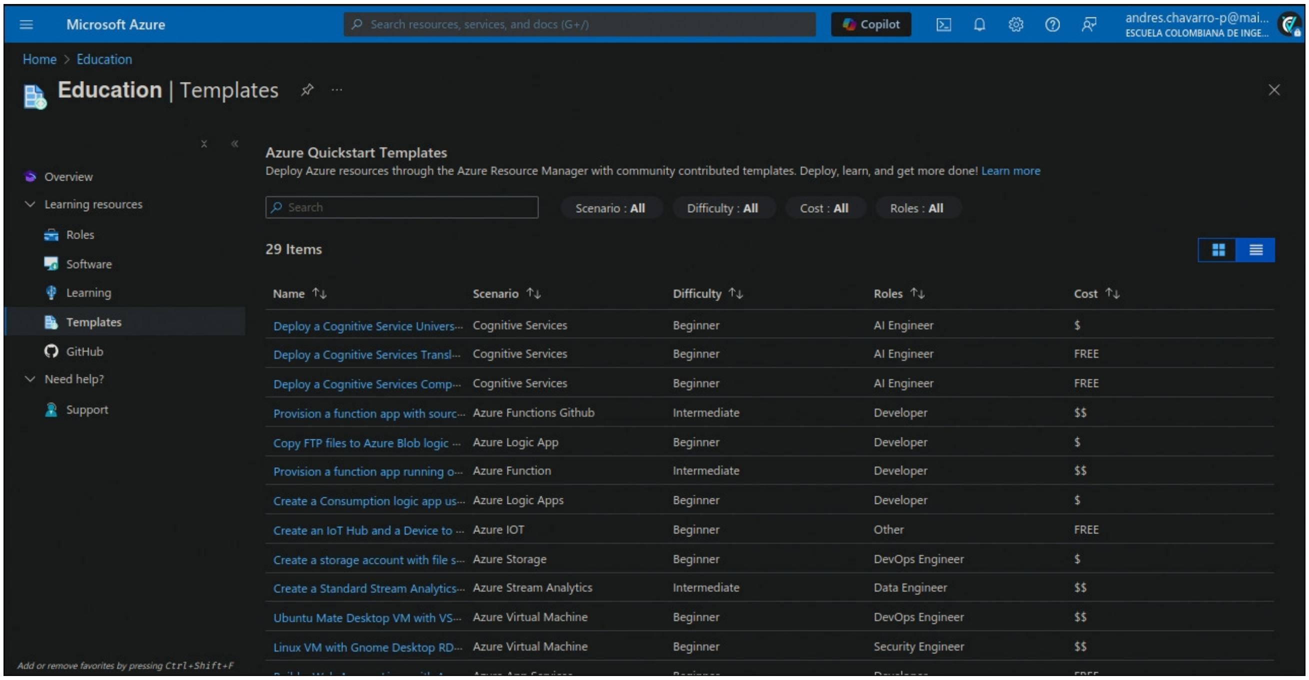
- Log in to the Azure portal.



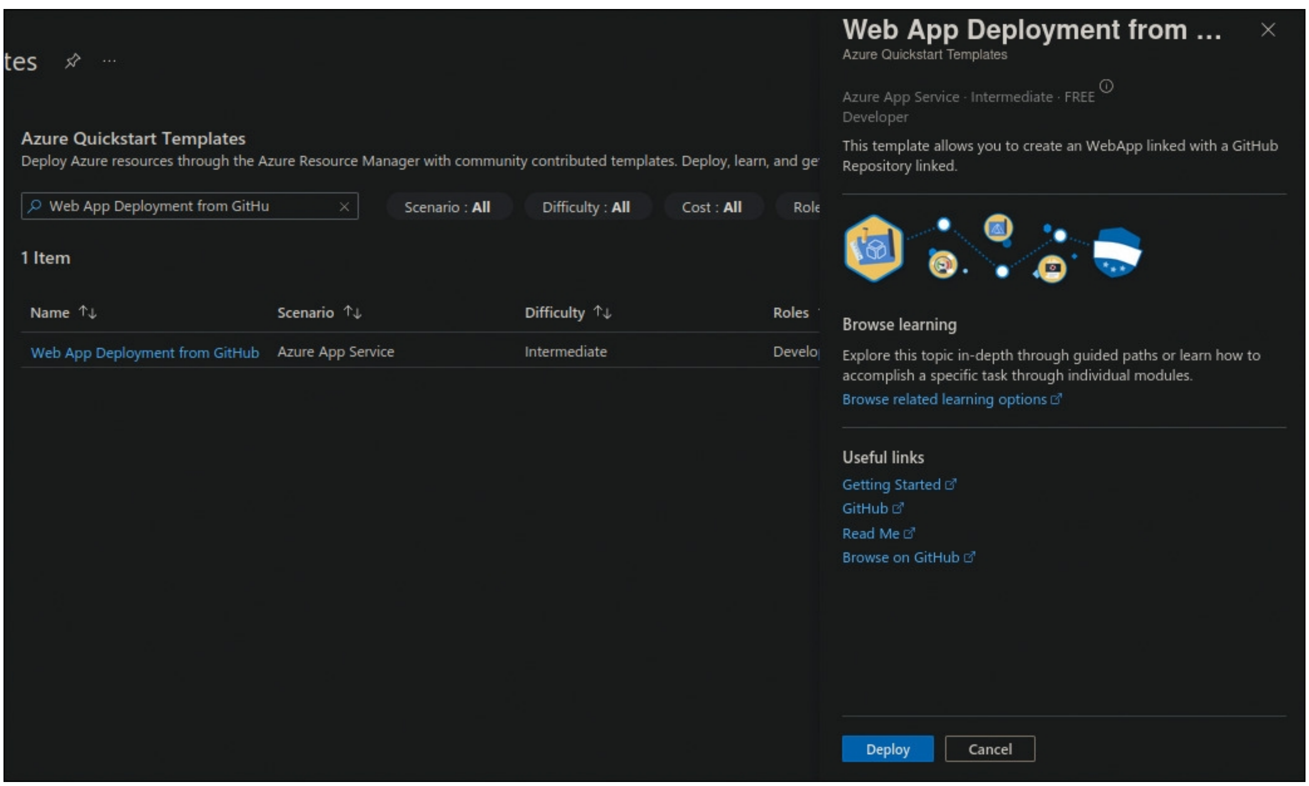
- Navigate to the "Education" section.



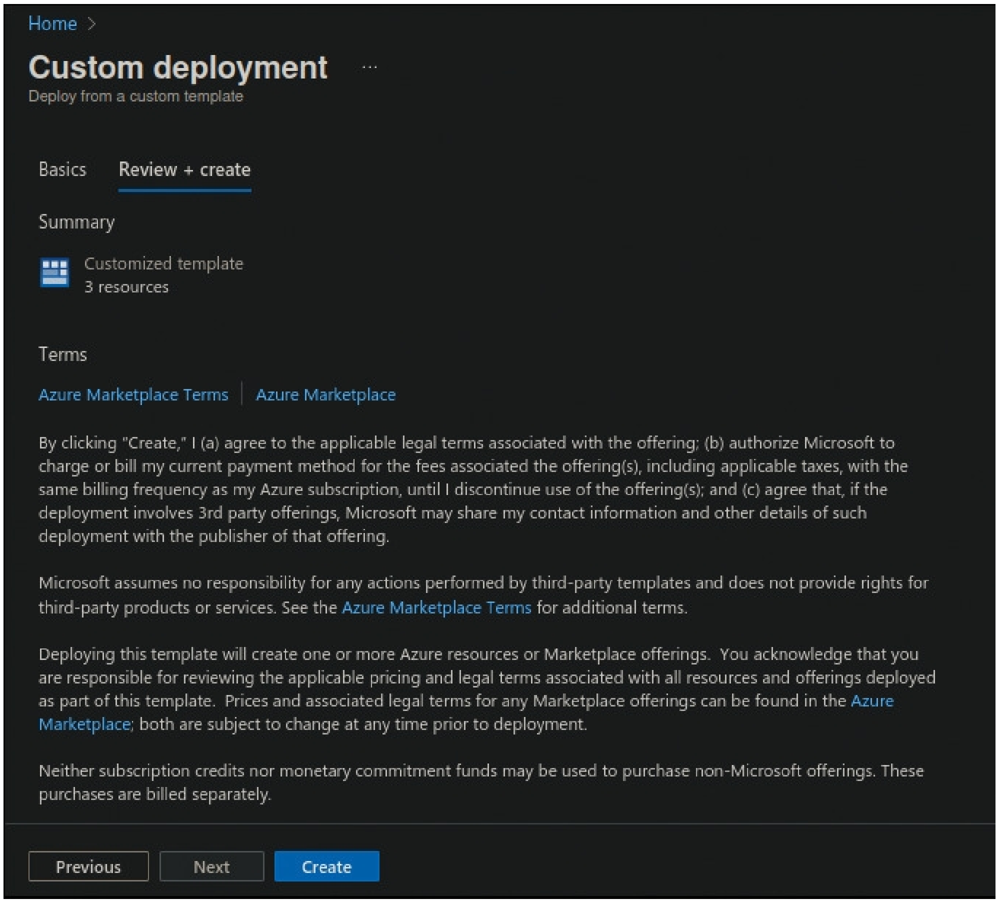
- Go to the ”Templates” section.



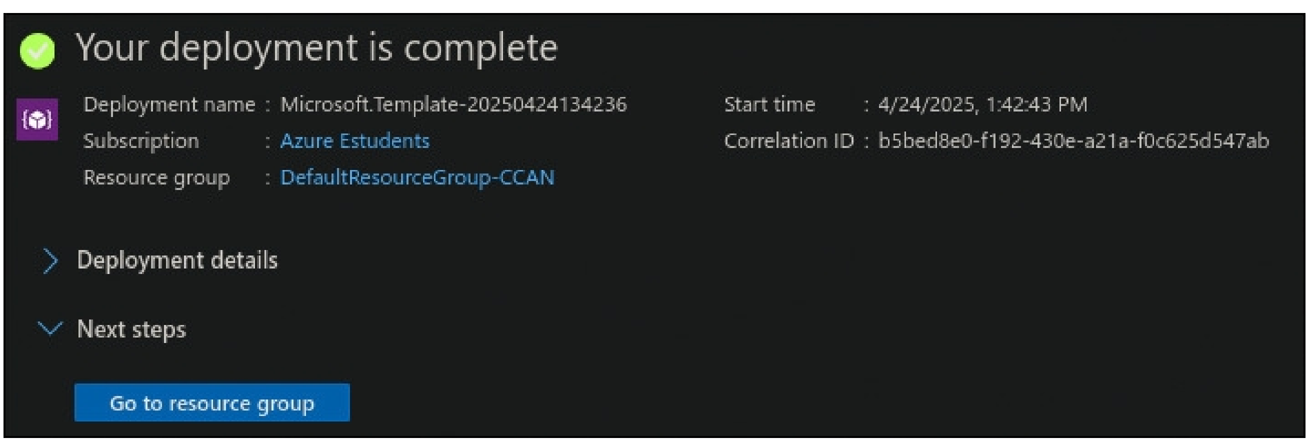
- Select the template named ”Web App Deployment from GitHub.”



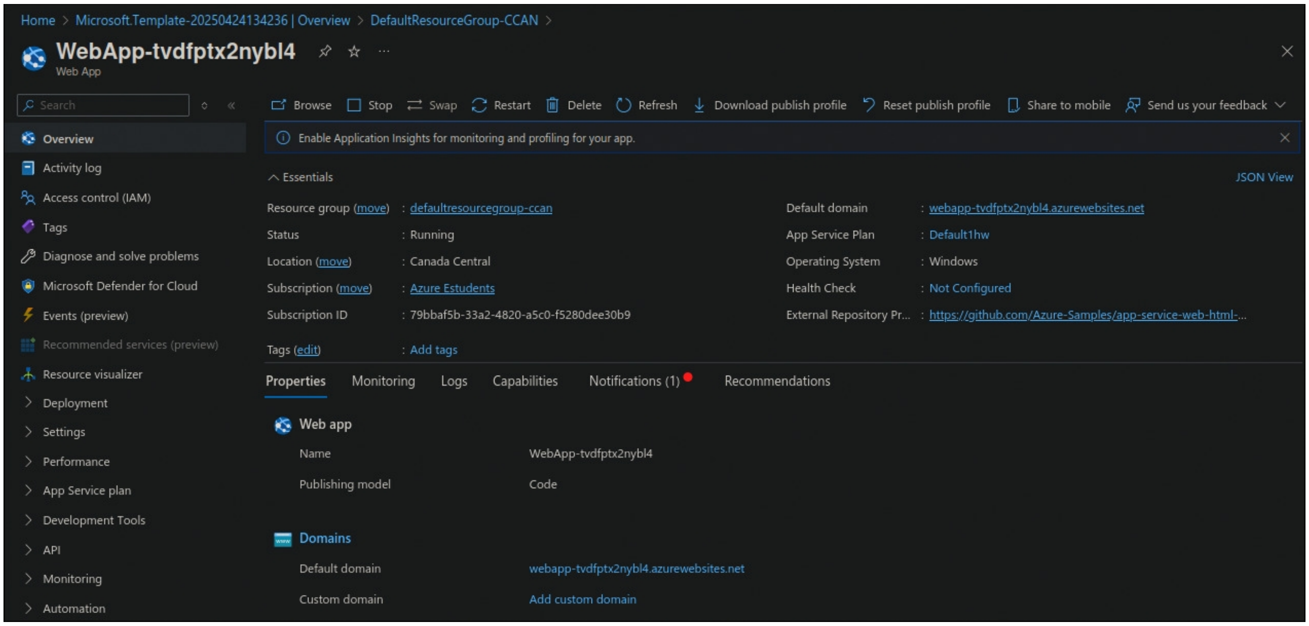
- Deploy the selected template.



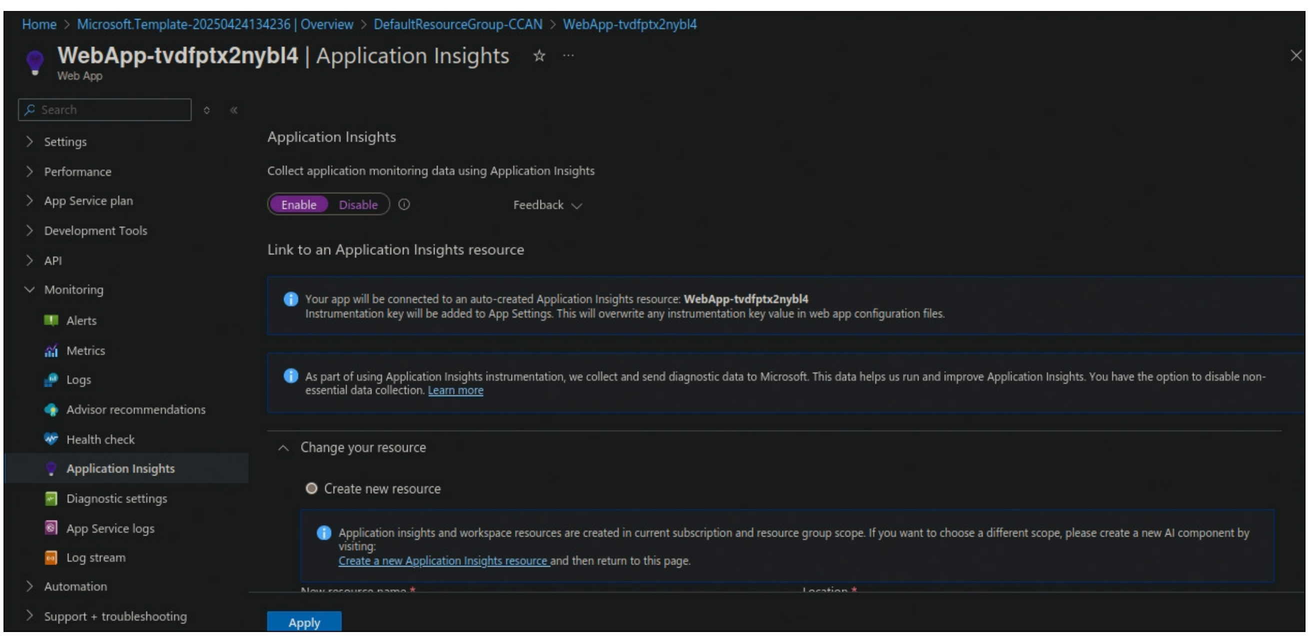
- Choose an existing resource or create a new one, then click ”Review + create” and subsequently ”Create.”



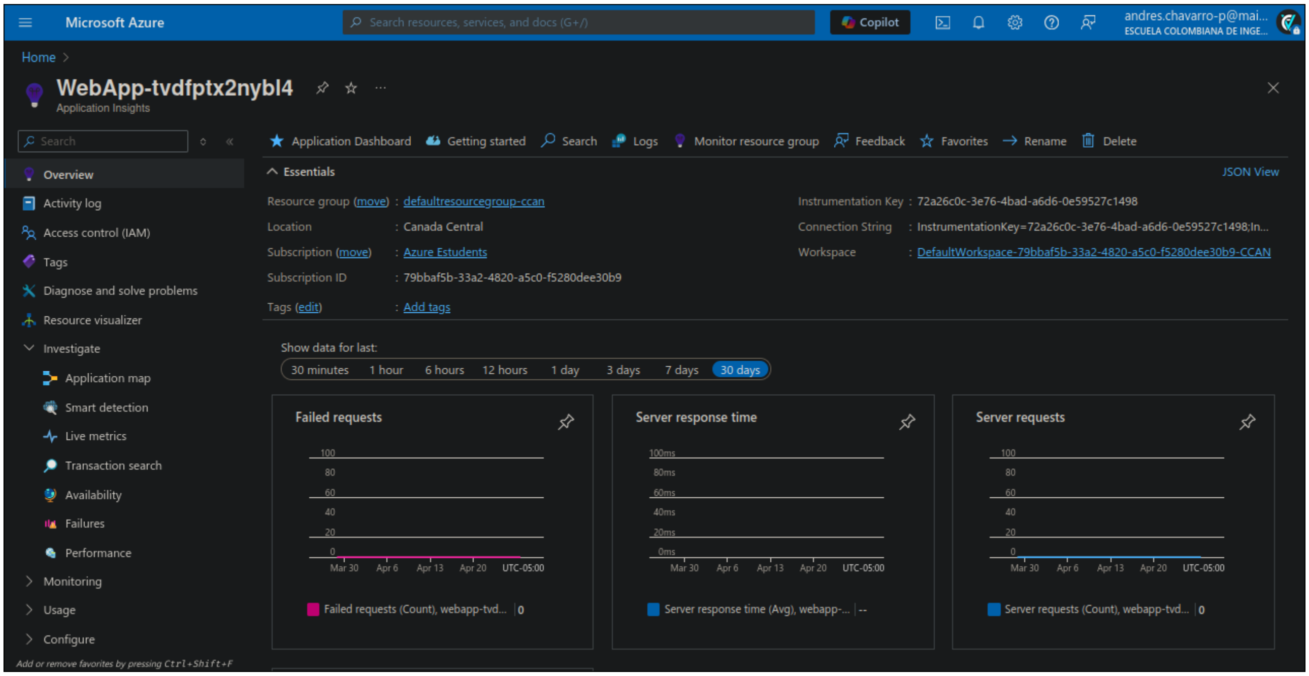
- Navigate to the created resource and access the ”Web App”. Explore the website by clicking the ”Default domain” at the ”Overview” section.

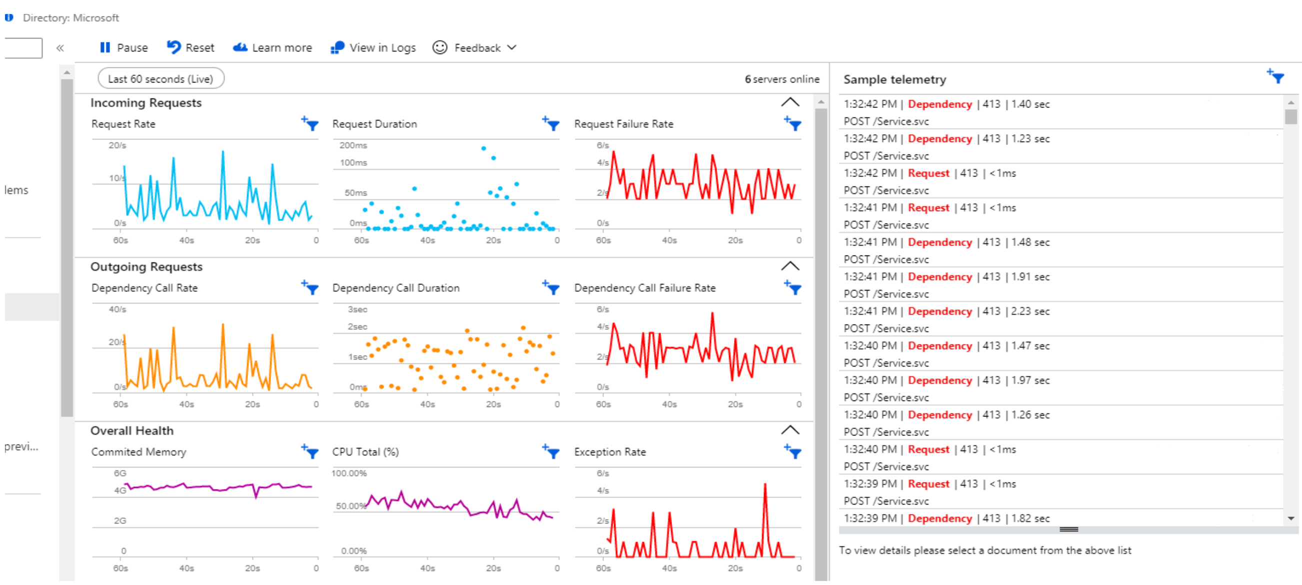


- Now, go to the ”Monitoring” section, then navigate to ”Application Insights” and enable the service if it isn’t already active. Once there, click on ”View Application Insights data.”



- Explore the "Overview" and "Live Metric" tabs in the "Investigate" section.





- What do you observe when you refresh the website repeatedly?

In these metrics tabs, we can see real-time information about requests, traffic, status, and other elements related to the web application. This way, it's possible to analyze behavior and determine if it's what we should or are looking for in our service.What does each of the items displayed there represent?

- What does each of the items displayed there represent?
    - HTTP Requests: The number of requests sent to the server.
    - Response time: The average time it takes the server to process a request and send a response.
    - HTTP status codes: 200 is successful response, 404 is resource not found, 500 is internal server error.
    - Dependencies: Time and success of connections to external services.
    - Availability rate: Percentage of requests that have been served correctly.
    - Resource usage: CPU, memory and storage monitoring.

- What other functionalities does the ”Application Insights” system offer for web services, databases, and other cloud-deployed systems?

  It depends greatly on the type of service being run. In the case of a Web Service, they allow us to analyze application performance in real time, access diagnostic tools for system health, generate alarms or notifications in the event of any failure, outage, or request to the user controlling the service, and other actions related to resource management for the service.

  On the other hand, databases use Insights to monitor queries that experience delays or irregularities and require administrator intervention. It also allows for understanding and validating the database's transition with different applications.

  Finally, for other deployed cloud services like AWS or Cloud, it gives us an overview of how some internal services within the same cloud are communicating or interfacing. At the same time, we have complete control over the cloud we're using and a large diagnostic center.

- How might this functionality be beneficial in a corporate environment?

  The functionality provided by systems like Application Insights can be highly beneficial in a corporate environment. By continuously monitoring applications and networks, organizations can quickly identify and resolve bottlenecks, improving overall system performance. This proactive approach helps maintain a positive customer experience by detecting and correcting issues before they impact users. Additionally, the detailed metrics collected allow companies to optimize their resource usage, leading to significant cost reductions. Automation features, such as alert configurations and automatic responses, further enhance system reliability by mitigating critical failures in real time. Finally, the rich data and comprehensive reports generated enable informed, data-driven decision-making, allowing companies to strategically guide their development, maintenance, and operational efforts.

- How does the network layer contribute to the transmission of data between the web app and clients in different geographic locations?

  The network layer plays a critical role in the transmission of data between a web application and clients located in different geographic regions. It is responsible for selecting optimized routes, ensuring that data travels through the most efficient path, even across continents. Additionally, the network layer can enable load balancing by redirecting traffic to servers that are either geographically closer to the client or experiencing less congestion, which improves response times and reliability.

- In what ways do the application and transport layers ensure reliable data transfer and correct interpretation of web app responses?

  The application and transport layers work together to ensure reliable data transfer and correct interpretation of web app responses. At the transport layer, TCP plays a key role by providing a reliable connection through acknowledgments and retransmitting any lost packets. It also segments data and reorders it at the destination to maintain integrity, while implementing flow and congestion control to prevent server overloads. Meanwhile, at the application layer, protocols like HTTP and HTTPS define a structured format for communication between client and server. HTTPS adds an extra layer of security through TLS/SSL encryption. Additionally, the use of standard methods like GET, POST, PUT, and DELETE ensures that client operations are clearly interpreted and correctly executed.

EXPERIMENTS

In this section, we conducted tests using ICMP messages, explored traceroute tools, and configured real routers with static routes to verify network communication.

USE OF ICMP MESSAGES

In this section we analyze the behavior of routers using console and graphical tools that implement the `***traceroute***` command.

a) We use the page given in the guide to see the routing for the two requested pages.

- Standford University:

Hop	Hostname	IP	AS	Network	Country	Avg (ms)
1	2600:3c0f:4::888	2600:3c0f:4::888	AKAMAI-LINODE-AP Akamai Connected Cloud, SG	2600:3c0f:4::/48		
2	2600:3c0f:4:35::14	2600:3c0f:4:35::14	AKAMAI-LINODE-AP Akamai Connected Cloud, SG	2600:3c0f:4::/48		
3	2600:3c0f:4:32::2	2600:3c0f:4:32::2	AKAMAI-LINODE-AP Akamai Connected Cloud, SG	2600:3c0f:4::/48		
4	Linode - Atlanta, GA	2600:3c02:100::106	AKAMAI-LINODE-AP Akamai Connected Cloud, SG	2600:3c02::/32		
5	???	???				
6	???	???				
7	???	???				
8	???	???				
9	e0-33.core1.las1.he.net	2001:470:0:3f3::2	HURRICANE, US	2001:470::/32		
10	???	???				
11	???	???				
12	???	???				
13	losa4-agg-01--lax-agg10--400g--01.cenic.net	2607:f380::118:9a40:b6f1	CENIC-2152, US	2607:f380::/32		
14	snvl2-agg-01--microsoft--100g--01.cenic.net	2607:f380::118:9a40:b151	CENIC-2152, US	2607:f380::/32		
15	dc-stanford--snvl2-agg-01-100ge.cenic.net	2607:f380:1::118:9a41:7911	STANFORD-INTERNET-ACCESS, US	2607:f380:1::118:9a41:7900/123		
16	campus-nw-rtr-vl1100.SUNet	2607:f6d0:0:1::12c1	STANFORD, US	2607:f6d0::/32		
17	woa-srtr-e1-25-vl3808.SUNet	2607:f6d0:0:1::22	STANFORD, US	2607:f6d0::/32		
18	web.stanford.edu	2607:f6d0:0:925a::ab43:d7c8	STANFORD, US	2607:f6d0::/32		

- Computer Science Laboratory:

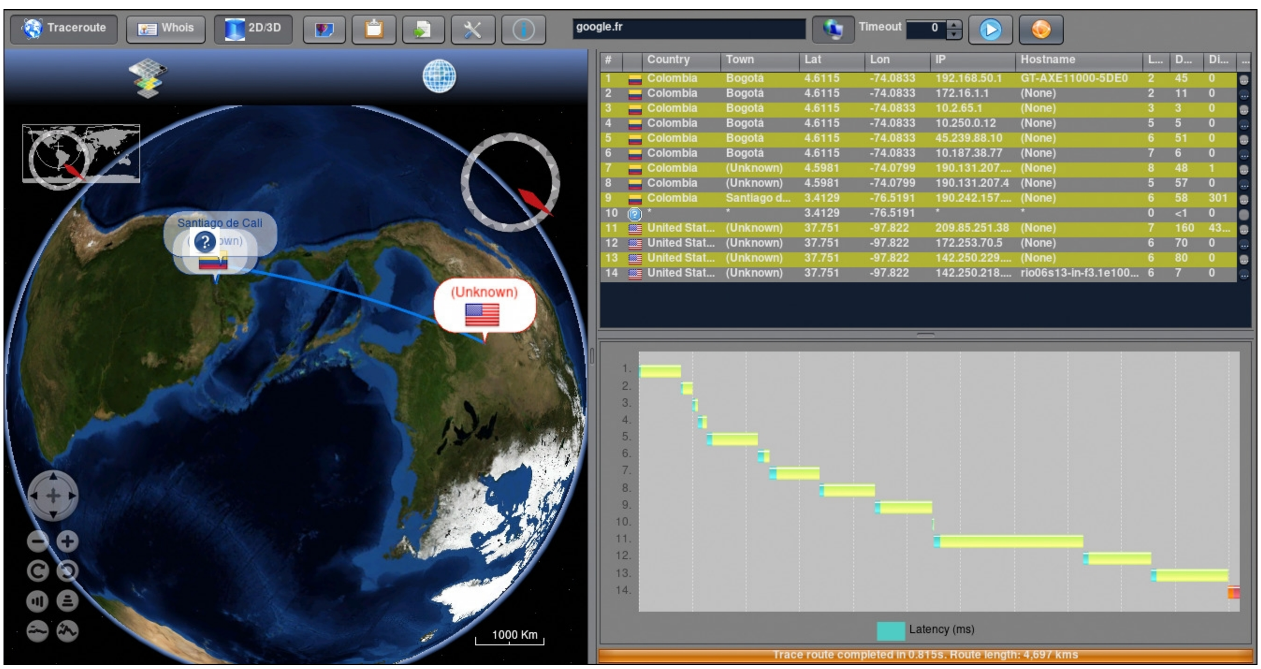
Hop	Hostname	IP	AS	Network	Country	Avg (ms)
1	10.204.3.120	10.204.3.120				
2	10.204.35.14	10.204.35.14				
3	10.204.32.2	10.204.32.2				
4	lo0-0.gw4.atl1.us.linode.com	74.207.239.106	AKAMAI-LINODE-AP Akamai Connected Cloud, SG	74.207.224.0/20		
5	ae45.r22.atl01.ien.netarch.akamai.com	23.203.144.36	AKAMAI-ASN1, NL	23.203.144.0/20		
6	atl-b24-link.ip.twelve99.net	62.115.171.94	TWELVE99 Arelion, fka Telia Carrier, SE	62.115.0.0/16		
7	mia-b2-link.ip.twelve99.net	62.115.113.49	TWELVE99 Arelion, fka Telia Carrier, SE	62.115.0.0/16		
8	boca-b3-link.ip.twelve99.net	62.115.118.162	TWELVE99 Arelion, fka Telia Carrier, SE	62.115.0.0/16		
9	asurnet-ic-383153.ip.twelve99-cust.net	213.248.100.171	TWELVE99 Arelion, fka Telia Carrier, SE	213.248.64.0/18		
10	ae0.col-ctx-mx03.cartagena.colombia.libertynetworks.com	69.79.100.53				
11	ae18.wbpcn-mx02.bogota.colombia.libertynetworks.com	69.79.100.77				
12	190.131.207.5	190.131.207.5	LIBERTY NETWORKS DE COLOMBIA S.A.S, CO	190.131.207.0/24		
13	190.131.207.183	190.131.207.183	LIBERTY NETWORKS DE COLOMBIA S.A.S, CO	190.131.207.0/24		

5	ae45.r22.atl01.ien.netarch.akamai.com	23.203.144.36	AKAMAI-ASN1, NL	23.203.144.0/20		
6	atl-b24-link.ip.twelve99.net	62.115.171.94	TWELVE99 Arelion, fka Telia Carrier, SE	62.115.0.0/16		
7	mia-b2-link.ip.twelve99.net	62.115.113.49	TWELVE99 Arelion, fka Telia Carrier, SE	62.115.0.0/16		
8	boca-b3-link.ip.twelve99.net	62.115.118.162	TWELVE99 Arelion, fka Telia Carrier, SE	62.115.0.0/16		
9	asurnet-ic-383153.ip.twelve99-cust.net	213.248.100.171	TWELVE99 Arelion, fka Telia Carrier, SE	213.248.64.0/18		
10	ae0.col-ctx-mx03.cartagena.colombia.libertynetworks.com	69.79.100.53				
11	ae18.wbpcn-mx02.bogota.colombia.libertynetworks.com	69.79.100.77				
12	190.131.207.5	190.131.207.5	LIBERTY NETWORKS DE COLOMBIA S.A.S, CO	190.131.207.0/24		
13	190.131.207.183	190.131.207.183	LIBERTY NETWORKS DE COLOMBIA S.A.S, CO	190.131.207.0/24		
14	???	???				
15	45.239.88.78	45.239.88.78	ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO, CO	45.239.88.0/22		
16	45.239.88.88	45.239.88.88	ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO, CO	45.239.88.0/22		

b) The tool seen can be emulated in the console using the `traceroute` command, as this gives us information about the routers sending data to obtain a response.

```
C:\Users\Redes>tracert google.fr

Tracing route to google.fr [142.250.218.99]
over a maximum of 30 hops:

  1    <1 ms    <1 ms    <1 ms  10.2.65.1
  2     1 ms     1 ms     1 ms  10.250.0.12
  3     2 ms     1 ms     2 ms  45.239.88.10
  4     2 ms     3 ms     2 ms  10.187.38.77
  5     3 ms     3 ms     2 ms  190.131.207.182
  6     2 ms     2 ms     1 ms  190.131.207.4
  7     4 ms     8 ms     4 ms  190.242.157.13
  8     3 ms     2 ms     3 ms  192.178.87.239
  9     2 ms     2 ms     2 ms  172.253.69.133
 10     3 ms     3 ms     2 ms  bog03s01-in-f3.1e100.net [142.250.218.99]

Trace complete.
```

c) We will use Open Visual Traceroute to analyze each different network and have a more complete view for routing analysis.

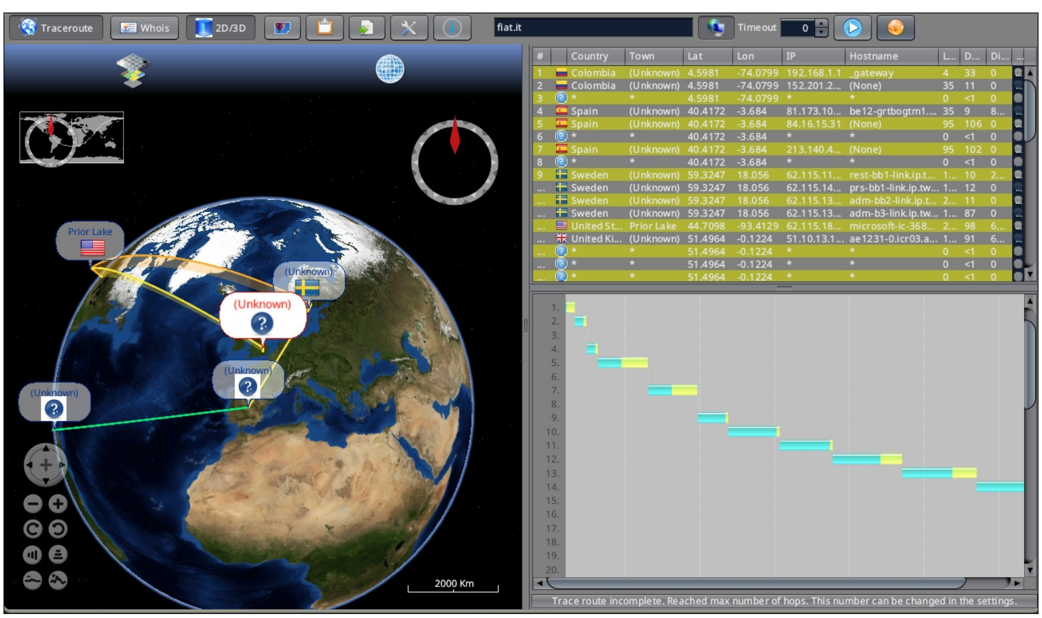


d) When trying the tool and seeing the options offered in the routing analysis, we could highlight the soup points:

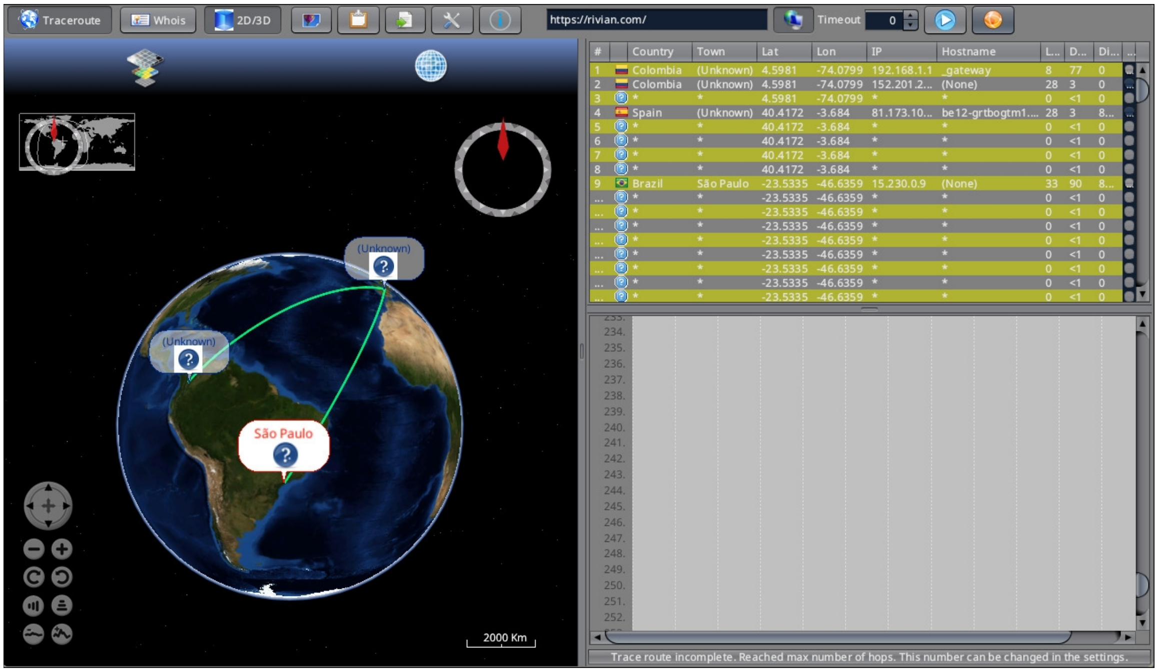
- Offers an intuitive and easy-to-navigate design, allowing users to efficiently analyzze routes and explore detailed information for each hop.
- Visually Maps The Journey of Packets Across Network Nodes, Helping Users Quickly Understand The Flow of Data.
- Measures The Latency At Each Hop and Identifies Points of Packet Loss, Assisting in Diagnosing Network Bottlenecks or Connectivity Issues.
- Provides The Physical Geographic Location of Each Node Along The Traced Path, Offering Greater Insight into the Network's Physical Distribution and Structure.
- Inable The Adjustment of Trace Paramers, Such As The Maximum Number of Hops and Packet Dispatch Intervals, Allowing Flexibility for Varyus Network Conditions.
- IPV4 and IPv6 support, it is full compatible with soh Major Internet Protocol Versions, Making It Versatile for Different Types of Network Environment.

e) Finally, we will use Open Visual Traceroute to view the routing of 5 different pages corresponding to car manufacturing areas.

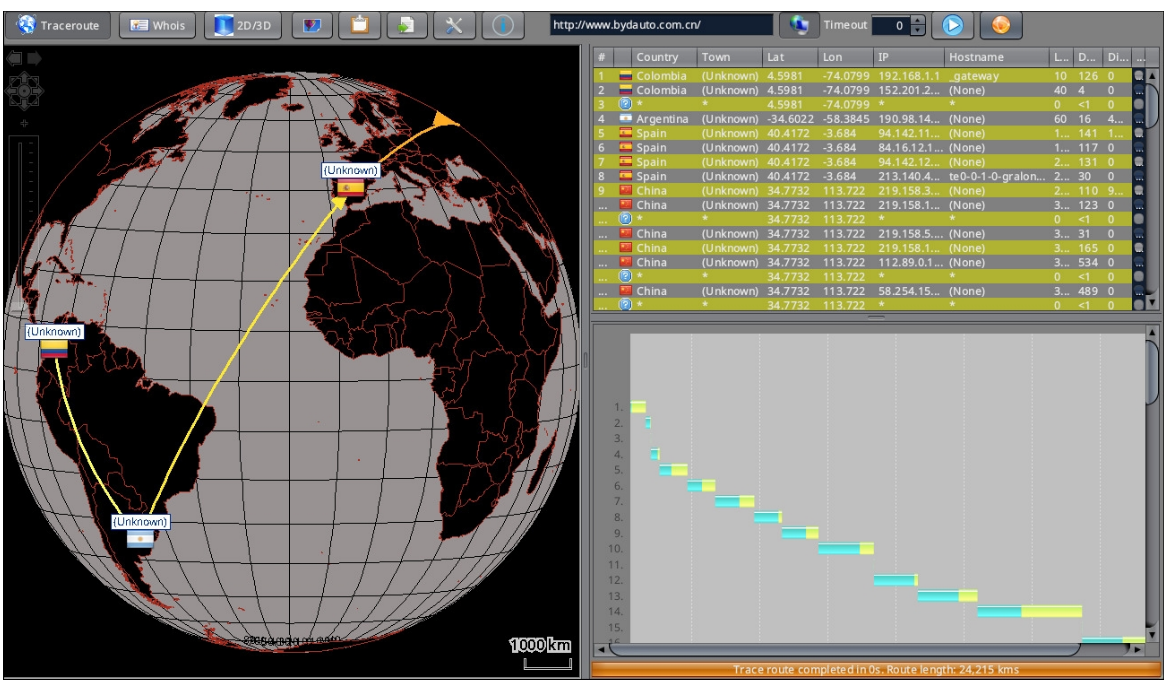
- http://fiat.it/



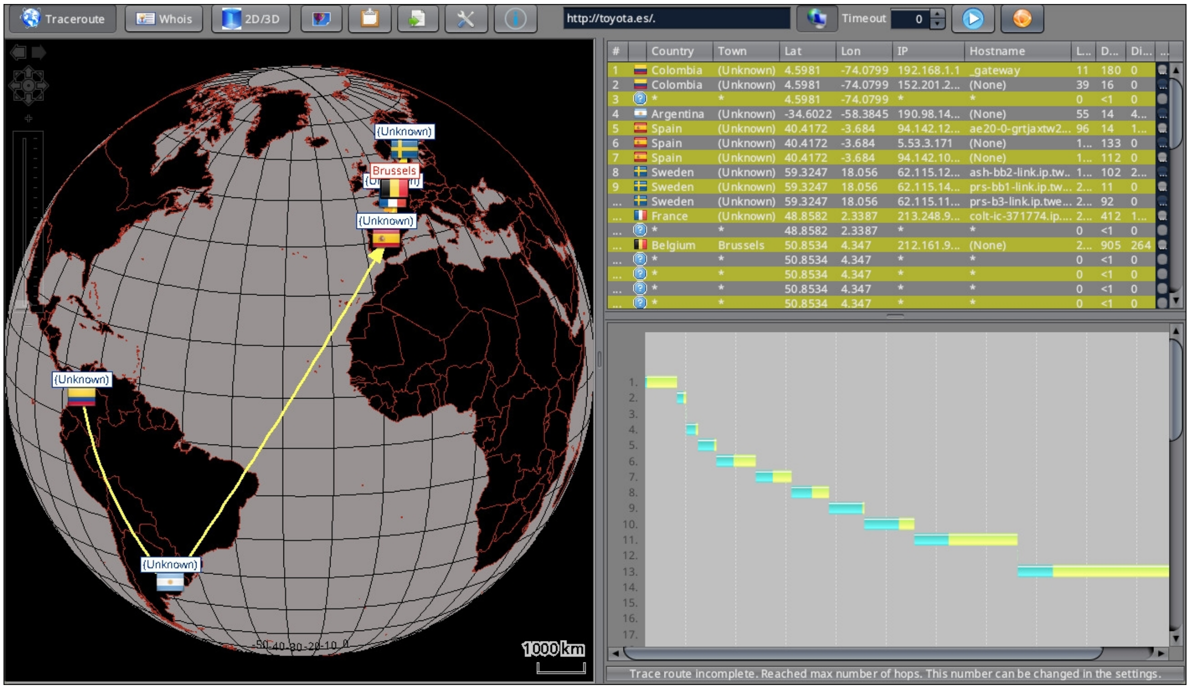
• https://rivian.com/



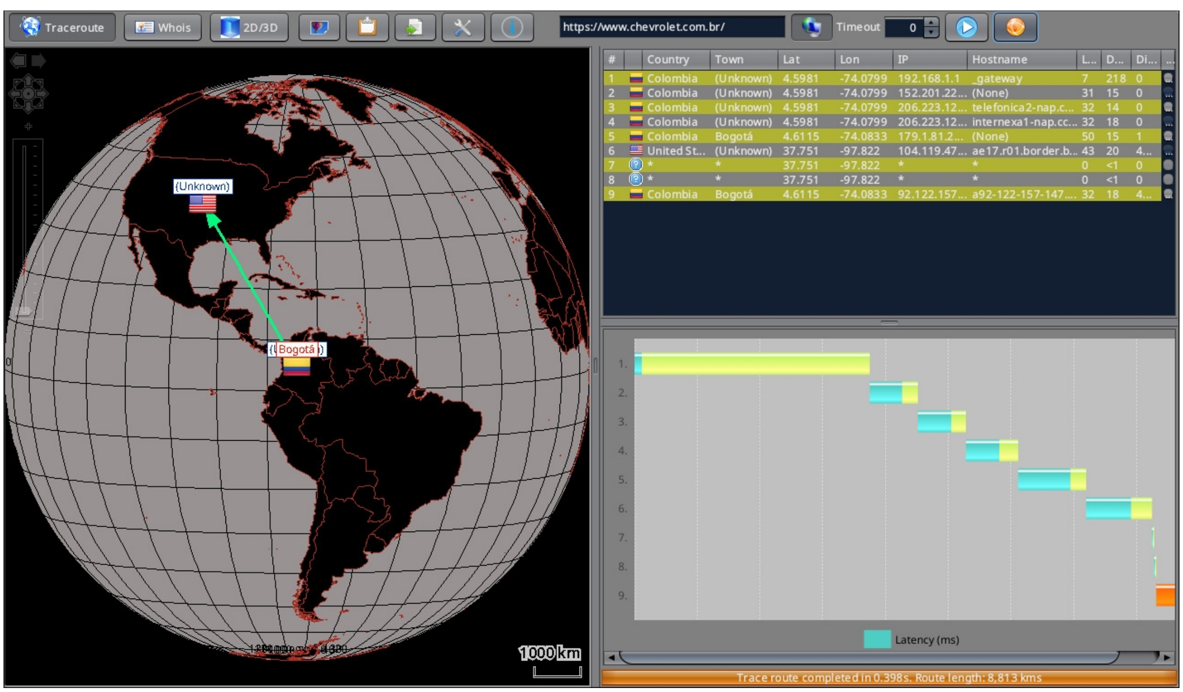
• http://www.bydauto.com.cn/



- http://toyota.es/



- https://www.chevrolet.com.br/



SOME QUESTIONS ABOUT ROUTER COMMANDS

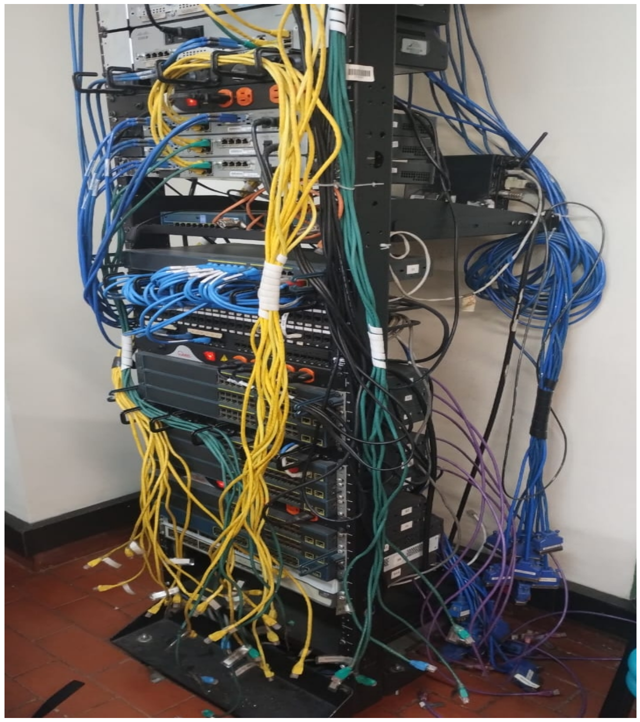
In this part of the lab, we have to review key concepts related to router configuration by answering fundamental questions about commands and internal processes.

1) Both commands protect access to privileged EXEC mode, but enable secret uses encryption while enable password does not. If both are set, the router gives precedence to enable secret.
2) The console is the physical port used for direct access to the router, typically via a cable connection. VTY known as Virtual Teletype, is used for remote access via Telnet or SSH.
3) The router loads the POST (Power-On Self-Test), then the bootstrap program, followed by the IOS image from flash memory. Finally, it loads the startup configuration from NVRAM, if available.
4) Types of memory in the routers:
   - ROM: Stores the bootstrap and diagnostic software.
   - RAM: Holds the running configuration and routing tables.
   - NVRAM: Stores the startup configuration.
   - Flash: Stores the IOS image.
5) The startup-configuration is the saved configuration loaded when the router boots. The running-configuration is the active configuration currently in use, which can be modified in real time.

SETUP - ACCESS AND BASIC CONFIGURATION OF ROUTERS

In this first section, we'll interact with the routers and infrastructure in the computer labs. Following the provided guide, we'll go step by step, answering or documenting the requested sections, as there's information to provide context for what was done in the lab.

First, we need to identify the infrastructure we'll be working with, as each router has its own characteristics that make managing its physical and virtual configurations slightly different. In short, we have a rack with the following types of routers:



In our case **we had router 2**, which has the following characteristics:

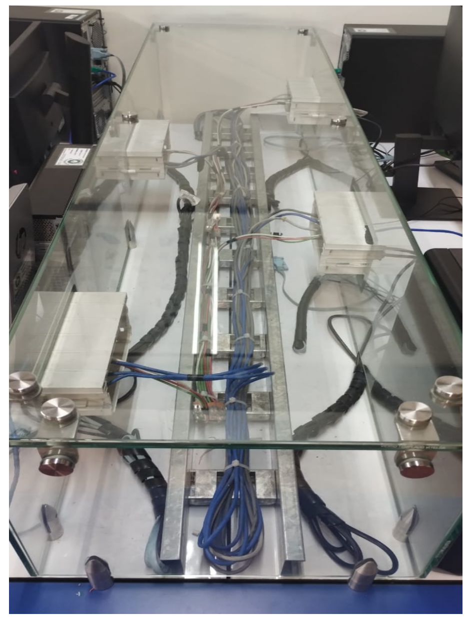
- Model: Cisco 1841 Integrated Services Router
- Category: Enterprise branch router
- Connectivity: 2 Fast Ethernet ports, 2 WAN interface card (WIC) slots, and support for advanced modular expansion.
- Capabilities: Provides secure data communication, VPN support, basic firewall capabilities, and flexible service integration.

Obviously there were also other types of routers, such as the 1941 series, which had similar features.

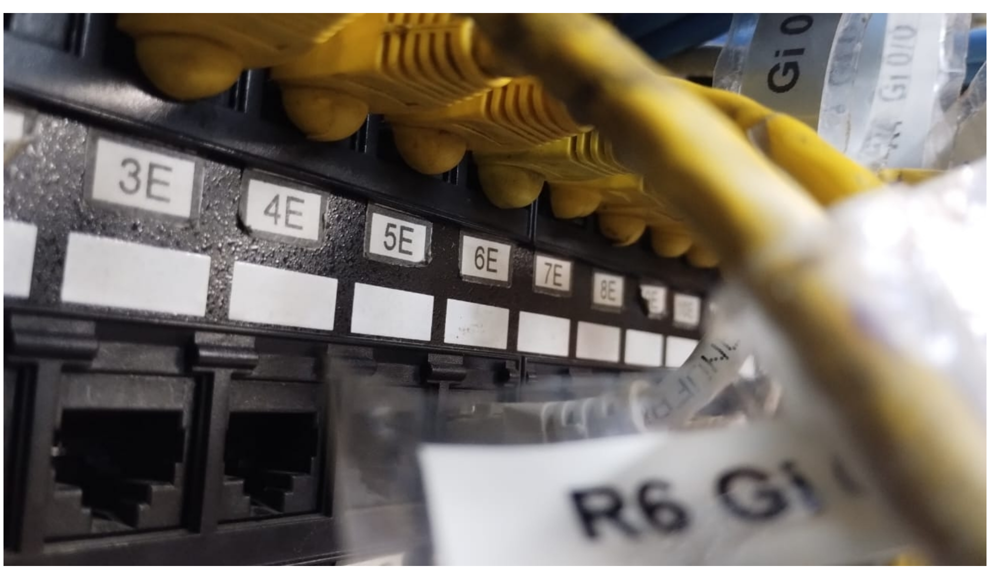
- Model: Cisco 1941 Integrated Services Router (ISR)
- Category: Enterprise branch router
- Connectivity: 2 integrated Gigabit Ethernet ports, 2 enhanced high-speed WAN interface card (EHWIC) slots.
- Capabilities: Supports advanced security (VPN, firewall), voice services, and application optimization

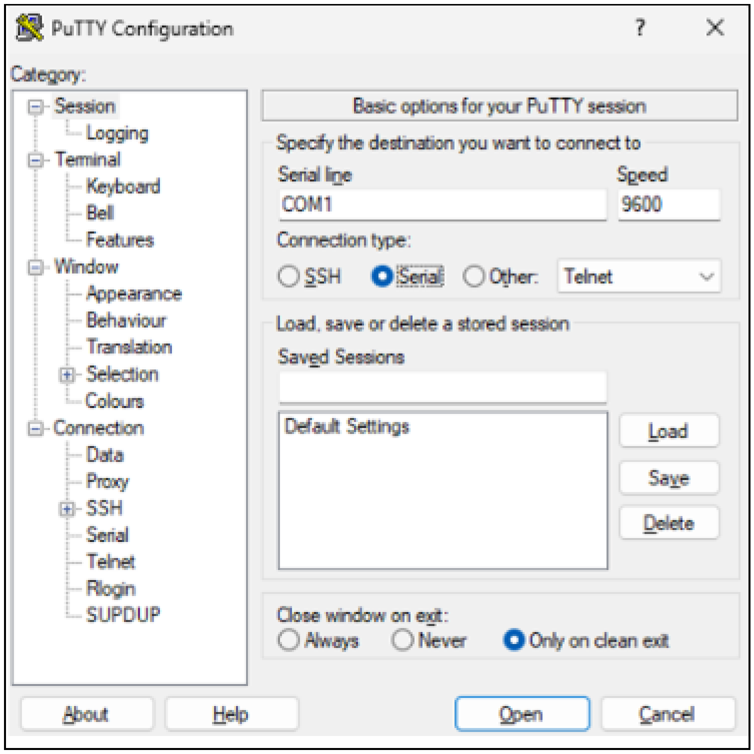
Now we'll connect the routers via console, so we can use a terminal emulator that connects via SSH, like PuTTY. This will give us access to the router and allow us to begin configuring it from our workstations.

We start by connecting our machines to the network cable located on the back. Using one of these cables, we can go to the router and place our respective cables in the switches.



In the rack, we need to find the cables for router 2 and connect them to the switch that has the same label as the connection point on the back of our devices, in our case, E5 and E6. Then, we should be able to connect PuTTY.





Before continuing, we need to understand the process the router goes through on boot when configured in modes 0x2142 and 0x2102, which are explained below.

When a Cisco router boots with the configuration register set to 0x2102, it follows the normal boot process: first, it executes POST to verify the hardware, then it locates and loads the IOS from flash memory, and finally, it loads the configuration file stored in NVRAM (startup-config) to apply the previously saved configurations. This is the standard operating mode in which the router boots with all its configuration.

On the other hand, if the router boots with the configuration register set to 0x2142, it ignores the configuration file saved in NVRAM. This allows the router to boot without loading previous passwords or configurations, which is very useful for password recovery tasks or entering initial configuration mode. The iOS loads normally, but boots into Setup Mode, allowing administrators to reconfigure the device or restore settings.

The main difference between 0x2102 and 0x2142 mode is the location from which the IOS is loaded. In 0x2102 mode, the router attempts to load the IOS from flash, and if it cannot find it, it attempts to load it from NVRAM. In 0x2142 mode, the router always loads the IOS from ROM.

Now we'll access the router and configure the settings outlined in the guide. This will give us a router with the ideal tools and configurations for the following actions in the lab. Keep in mind that the image shows all the settings in sequence.

```
Router>
Router>
Router>
Router>
Router>enable
Router#configure terminal Espinosa
                          ^
% Invalid input detected at '^' marker.

Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname Espinosa
Espinosa(config)#enable
% Incomplete command.

Espinosa(config)#clear
                   ^
% Invalid input detected at '^' marker.

Espinosa(config)#show version
                   ^
% Invalid input detected at '^' marker.

Espinosa(config)#enable password cisco
Espinosa(config)#line console 0
Espinosa(config-line)#password claveC
Espinosa(config-line)#login
Espinosa(config-line)#logging synchronouns
Translating "synchronouns"...domain server (255.255.255.255)
                                        ^
% Invalid input detected at '^' marker.

Espinosa(config-line)#logging synchronous
Espinosa(config-line)#exit
Espinosa(config)#line vty 0 4
Espinosa(config-line)#password claveT
Espinosa(config-line)#login
Espinosa(config-line)#logging synchornous
Translating "synchornous"...domain server (255.255.255.255)
                                   ^
% Invalid input detected at '^' marker.

Espinosa(config-line)#logging synchronous
Espinosa(config-line)#exit
Espinosa(config)#no ip domain-lookup
Espinosa(config)#banner motd #Buenas, practica lab 07#
Espinosa(config)#
```

```
Espinosa(config)#show version
                    ^
% Invalid input detected at '^' marker.

Espinosa(config)#enable password cisco
Espinosa(config)#line console 0
Espinosa(config-line)#password claveC
Espinosa(config-line)#login
Espinosa(config-line)#logging synchronouns
Translating "synchronouns"...domain server (255.255.255.255)
                                          ^
% Invalid input detected at '^' marker.

Espinosa(config-line)#logging synchronous
Espinosa(config-line)#exit
Espinosa(config)#line vty 0 4
Espinosa(config-line)#password claveT
Espinosa(config-line)#login
Espinosa(config-line)#logging synchornous
Translating "synchornous"...domain server (255.255.255.255)
                                     ^
% Invalid input detected at '^' marker.

Espinosa(config-line)#logging synchronous
Espinosa(config-line)#exit
Espinosa(config)#no ip domain-lookup
Espinosa(config)#banner motd #Buenas, practica lab 07#
Espinosa(config)#interface GigabithB?
% Unrecognized command
```

Now we're going to interconnect the table devices by assigning them an IP address. However, this is obtained through a subnet to another general network that was given to us in the guide. The nice thing is that no matter what IP address is assigned, you can ping them and know that they are correctly connected.

```
Espinosa(config)#interface GigabitEthernet0/0
Espinosa(config-if)#description #Equipo de 600 hosts
Espinosa(config-if)#ip address 88.0.4.0 255.255.252.0
Bad mask /22 for address 88.0.4.0
Espinosa(config-if)#ip address 88.0.4.1 255.255.252.0
Espinosa(config-if)#interface GigabitEthernet0/1
Espinosa(config-if)#description #Equipo de 4000 hosts
Espinosa(config-if)#ip address 88.0.16.1 255.255.240.0
Espinosa(config-if)#no shutdown
Espinosa(config-if)#exit
Espinosa(config)#description #Equipo de 4000 hosts
*Apr  7 22:26:53.911: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to down
Espinosa(config)#
*Apr  7 22:27:02.151: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to up
*Apr  7 22:27:03.151: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
Espinosa(config)#interface GigabithEthernet0/0
                                   ^
% Invalid input detected at '^' marker.

Espinosa(config)#interface GigabitEthernet0/0
Espinosa(config-if)#description #Equipo de 600 hosts
Espinosa(config-if)#ip address 88.0.4.1 255.255.252.0
Espinosa(config-if)#no shutdown
Espinosa(config-if)#exit
Espinosa(config)#
*Apr  7 22:27:33.487: %LINK-3-UPDOWN: Interface GigabitEthernet0/0, changed state to down
Espinosa(config)#
*Apr  7 22:27:42.151: %LINK-3-UPDOWN: Interface GigabitEthernet0/0, changed state to up
*Apr  7 22:27:43.151: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
Espinosa(config)#
```

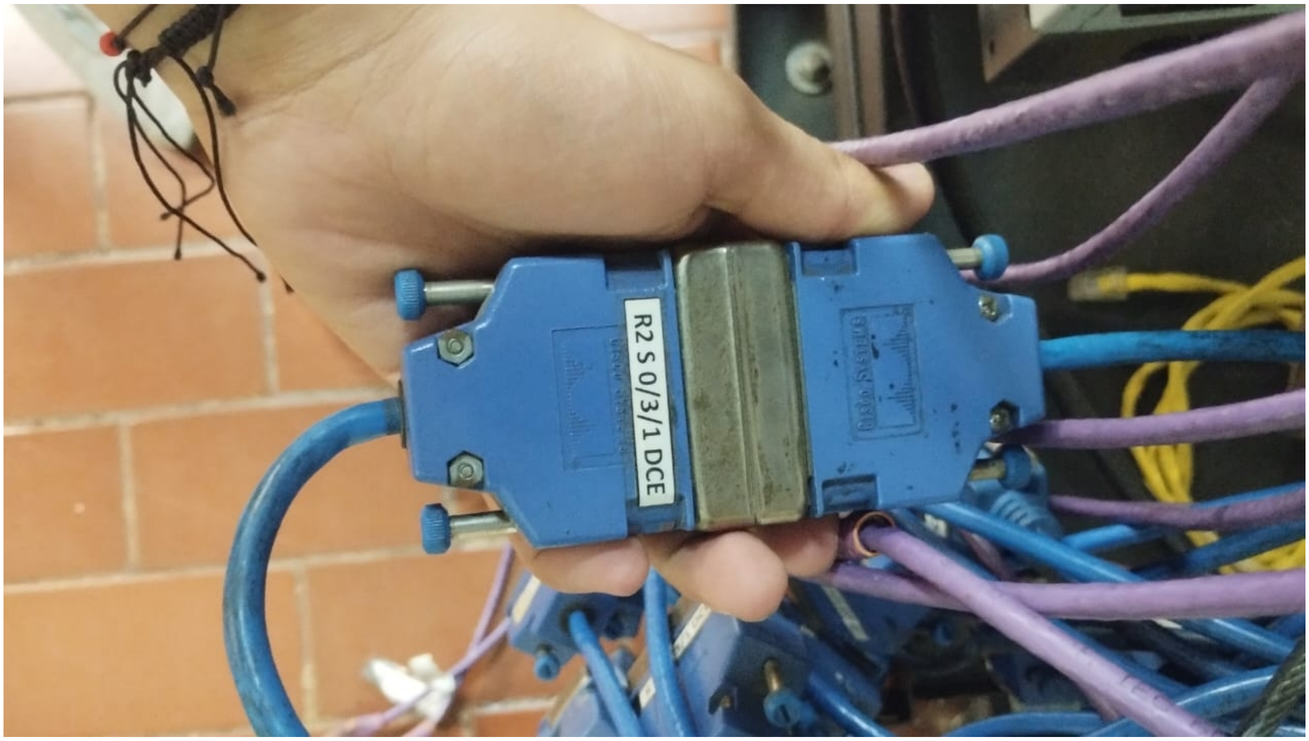
In this way, we can ping a machine that is also connected to the same router, if the connection and configuration are done correctly, there should be no problem.

```
Pinging 88.0.16.1 with 32 bytes of data:
Reply from 88.0.16.1: bytes=32 time<1ms TTL=255
Reply from 88.0.16.1: bytes=32 time<1ms TTL=255
Reply from 88.0.16.1: bytes=32 time<1ms TTL=255
Reply from 88.0.16.1: bytes=32 time<1ms TTL=255

Ping statistics for 88.0.16.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\Redes>

C:\Users\Redes>ping 88.0.4.2

Pinging 88.0.4.2 with 32 bytes of data:
Reply from 88.0.4.2: bytes=32 time=1ms TTL=127
Reply from 88.0.4.2: bytes=32 time=1ms TTL=127
Reply from 88.0.4.2: bytes=32 time=1ms TTL=127
Reply from 88.0.4.2: bytes=32 time=1ms TTL=127

Ping statistics for 88.0.4.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 1ms, Average = 1ms
```

```
C:\Users\Redes>ping 88.0.4.1

Pinging 88.0.4.1 with 32 bytes of data:
Reply from 88.0.4.1: bytes=32 time<1ms TTL=255
Reply from 88.0.4.1: bytes=32 time<1ms TTL=255

Ping statistics for 88.0.4.1:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
Control-C
^C
C:\Users\Redes>ping 88.0.16.2

Pinging 88.0.16.2 with 32 bytes of data:
Reply from 88.0.16.2: bytes=32 time<1ms TTL=127
Reply from 88.0.16.2: bytes=32 time<1ms TTL=127
Reply from 88.0.16.2: bytes=32 time=1ms TTL=127
Reply from 88.0.16.2: bytes=32 time<1ms TTL=127

Ping statistics for 88.0.16.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\Redes>
```

SETUP - SERIAL INTERCONNECTION

For this part, we have the equipment already prepared to be able to make interconnections with other devices. In this case, we are given a network that we have to divide with subnetting and each of the tables connect the peer computers that exist. In our case, this was the result of try ping each other computer. Before starting, we have to connect the routers with the serial cables, as shown in the photo, we connect router 2 and 1.



A null modem is a special type of cable that allows direct communication between two network devices or computers without the need for an actual modem. In serial connections between routers, a null modem allows this direct communication to be simulated by swapping the transmit and receive pins. The ping, are not working at the moment, we need routing the IP's.

The clock rate command on routers is used to define the transmission speed on a serial interface. It is essential when a router acts as a DCE to establish the frequency at which data is sent, as the DCE device always provides the synchronizing clock to the DTE.

Regarding the terms DTE (Data Terminal Equipment) and DCE, on a serial link, one of the devices must act as the DCE and provide the clock, while the other is the DTE and simply follows that clock.

In the network lab, when we connect two routers directly, one must be configured as a DCE, and the clock rate command must be applied on that device for communication to work properly.

STATIC ROUTING

In the previous part, there was a problem with pings, since there was no routing that allowed the connection between the routers or the equipment. To do this, we manually set each router according to the IP given by the teacher, resulting in the following in our case.

```
Buenas, practica lab 07

User Access Verification

Password:
Password:
Espinosa>enable
Password:
Espinosa#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Espinosa(config)#interface Serial0/0/0
Espinosa(config-if)#ip address 21.0.0.2 255.0.0.0
Espinosa(config-if)#clock rate 64000
%Error: This command applies only to DCE interfaces
Espinosa(config-if)#no shutdown
Espinosa(config-if)#exit
Espinosa(config)#
*Apr  7 23:10:37.267: %LINK-3-UPDOWN: Interface Serial0/0/0, changed state to up
*Apr  7 23:10:38.267: %LINEPROTO-5-UPDOWN: Line protocol on Interface Serial0/0/0, changed state to up
Espinosa(config)#ip route 87.0.4.0 21.0.0.1
% Incomplete command.

Espinosa(config)#ip route 87.0.4.0 255.255.252.0 21.0.0.1
Espinosa(config)#ip route 87.0.16.0 255.255.240.0 21.0.0.1
Espinosa(config)#end
Espinosa#wr
*Apr  7 23:11:49.355: %SYS-5-CONFIG_I: Configured from console by console
Espinosa#write memory
Building configuration...
[OK]
Espinosa#D
```

If we perform the test with pings, we see that the devices are already connected, and we could say that we have a small internet network between the computers in the laboratory.

```
Ping statistics for 87.0.4.1:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 18ms, Maximum = 18ms, Average = 18ms
Control-C
^C
C:\Users\Redes>ping 87.0.16.2

Pinging 87.0.16.2 with 32 bytes of data:
Reply from 87.0.16.2: bytes=32 time=18ms TTL=126
Reply from 87.0.16.2: bytes=32 time=18ms TTL=126

Ping statistics for 87.0.16.2:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 18ms, Maximum = 18ms, Average = 18ms
Control-C
^C
C:\Users\Redes>ping 87.0.4.2

Pinging 87.0.4.2 with 32 bytes of data:
Reply from 87.0.4.2: bytes=32 time=18ms TTL=126

Ping statistics for 87.0.4.2:
    Packets: Sent = 1, Received = 1, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 18ms, Maximum = 18ms, Average = 18ms
Control-C
^C
C:\Users\Redes>D
```

By repeating this process multiple times with the other working groups, we were able to complete the network. However, due to time constraints, this was not possible due to technical issues. Even so, we can say that this part is complete.

CLOSURE

To finish the exercise, we need to format the routers so they're clean and the configurations made in the previous steps aren't saved. Other groups require the space to perform their respective exercises.

To do this, we need to follow these steps in the PuTTY console, entering certain commands:

- Delete saved settings: Using the `erase startup-config` command we will be able to permanently delete existing configurations.
- Delete the VLAN database: just in case we try to delete this internally stored information, using the `delete vlan.dat` command.
- Reboot the router: Using the `reload` command and using the router's physical switch to turn it off and back on, we can reboot the device.

After rebooting, the router will return to its initial mode with the message: "*Would you like to enter the initial configuration dialog? [yes/no]:*" If we choose the "no" option, it will allow us to reconfigure the routers, as they have no saved configurations. This way, they are completely reset to their original configuration.

CONCLUTIONS

Throughout this lab, we developed a deeper understanding of how operating systems and network infrastructures work together by engaging in both virtual configurations with physical devices. We started by studying and scripting essential network commands like netstat, route, and vnstat, allowing us to analyze system network information and automate these tasks through custom Shell menus. This helped us become more familiar with network diagnostics and service monitoring.

We then installed and configured monitoring servers on different operating systems such as Slackware and NetBSD, using SNMP based tools to track the status and performance of our virtual machines. This part of the lab showed us how system administrators monitor networks in real time and handle resource usage, service availability, and performance metrics.

Additionally, deploying a web app in Microsoft Azure taught us how cloud platforms handle application hosting, live monitoring, and metrics visualization through tools like Application Insights. We explored how various layers of the network model work together to deliver reliable communication between users and cloud resources across different regions.

Lastly, we worked directly with routers, using console cables and software like PuTTY to configure them manually. We created access credentials, disabled unnecessary lookups, set hostnames, and enabled communication between PCs through static routing and serial interconnections. We also collaborated with other groups to expand the network and ensured full connectivity using ICMP tools like ping and traceroute.

By completing all of these tasks, we gained valuable practical experience in setting up and managing real-world network environments, reinforcing the theoretical concepts behind modern IT infrastructure.

BIBLIOGRAFY

Basic router configuration. (2017, febrero 5). Cisco. https://www.cisco.com/c/en/us/td/docs/routers/access/800M/software/800MSCG/routconf.html

Fernández, Y. (2020, julio 13). *Tracert o Traceroute: qué es, cómo funciona o cómo se utiliza*. Xataka.com; Xataka. https://www.xataka.com/basics/tracert-traceroute-que-como-funciona-como-se-utiliza

Fernández, Y. (2023, marzo 29). *Guía para configurar tu router a fondo: dónde colocarlo, cómo configurarlo y sus principales funciones*. Xataka.com; Xataka. https://www.xataka.com/basics/guia-router-todo-que-debes-saber-para-colocarlo-configurarlo-usar-sus-principales-funciones

Infante, D. C. H. (2023, mayo 31). *Comando Traceroute: cómo usarlo e interpretarlo*. Tutoriales Hostinger. https://www.hostinger.com/co/tutoriales/comando-traceroute

JnHs. (s/f). *Azure documentation*. Microsoft.com. Recuperado el 28 de abril de 2025, de https://learn.microsoft.com/en-us/azure/?product=popular

ManageEngine. (s/f). *Tutorial: Fundamentos de SNMP*. Manageengine.com. Recuperado el 28 de abril de 2025, de https://www.manageengine.com/latam/network-monitoring/tutorial-fundamentos-protocolo-snmp.html

¿Qué es SNMP? (s/f). Whatsupgold.com. Recuperado el 28 de abril de 2025, de https://www.whatsupgold.com/es/snmp

Small business IT explained in 60 seconds or less: Switches vs. Routers. (2019, agosto 28).

Your journey animated overview video. (2025, febrero 6).

(S/f). Cisco.com. Recuperado el 28 de abril de 2025, de https://www.cisco.com/c/dam/en/us/td/docs/routers/access/cisco_router_and_security_device_manager/24/software/user/guide/SDMH24.pdf